

Streaming Notes

OBS setup and workflow for streaming game development

VIDEO GAME CREATION

A running set of notes for streaming, kept separate from the Book Notes and the class materials. Add to it as the setup changes. Entries are in Robert's own words, lightly tidied.

Quick Reference

Engine	OBS source	Key setting
GameMaker	Game Capture	Window Capture will NOT capture gameplay.
Godot	Window Capture	Window Match Priority: Window Title Must Match. In the Play window's three-dot menu, uncheck Make Workspace Floating on next play.
Unity	Currently full-screen capture (to change)	Plan: separate OBS scenes for Unity and for Visual Studio.

Note 1: Capturing Play Test Windows in OBS

Several game engines do not embed their play-test mode in the IDE. For streaming, that leaves two choices: stream in full-screen capture, or create separate OBS scenes, each set to capture the correct window.

Current preference: avoid full-screen capture, and switch between OBS scenes as needed so that only the window you want is captured.

Process while streaming: switch to the right scene BEFORE starting a play test, so the stream captures the play window the moment it opens.

GameMaker

The source has to be a Game Capture. A Window Capture will not capture the gameplay.

Godot

Use a Window Capture, with Window Match Priority set to "Window Title Must Match". A second setting is needed inside Godot as well, because by default Godot floats the play window. That seems to create two windows with the same title, so OBS fails to grab the right one once the game window is closed and restarted.

Fix: in the Godot Play window, select the three dots in the upper right and uncheck "Make Workspace Floating on next play". This puts the play window inside the IDE (sort of), which

Robert prefers anyway, and makes the window consistently recognizable to OBS, which is what matters.

Standard Window Source Settings for Game Play Screens

Apply to the OBS window sources that capture a game's play window (not to editor or reference windows):

- Capture Audio: on, for game-playing screens only (the application audio route chosen in Note 4). Leave it off for reference windows such as Acrobat, Claude and the browser.
- Capture Cursor: on.
- Quirk: Capture Cursor only works while the captured window is selected (has focus). If you click out of the window and then move the mouse back over it, the effects of your actions still show (menus expanding, highlighting) but the pointer itself does not appear on stream until you click on the window. It makes sense once you think about it, but it is easy to miss: click the captured window once to bring the pointer back.
- Window Match Priority: "Match title, otherwise find window of same executable".
- Transform: Fit to Screen, so the captured window always displays as large as it can on the stream canvas while keeping its proportions (right-click the source in the scene, then Transform > Fit to Screen; Ctrl+F). Do not use Stretch to Screen, which distorts the picture. Set it on every window source, not just play screens, and re-check it if a window is resized.

Possible link to the Godot problem: the Window Match Priority setting may have been part of the original Godot play-test problem, alongside the floating-window issue. Godot was previously set to "Window Title Must Match" (Note 1 above). Test whether the new priority setting reattaches to a restarted Godot play window on its own, with and without the floating setting changed, and record which fix is actually needed.

Force SDR (a Window Capture option)

What it is: an option that makes OBS ignore the monitor's color space and treat the captured window as standard dynamic range (SDR) content. It exists for computers whose display is running in Windows HDR mode: without it, SDR content captured from an HDR display can look washed out or have wrong brightness. If Windows HDR is off, it should make no difference. Users report that for real HDR games it can blow out highlights and mid-range colors. It is off by default; it is not automatic and must be set by hand.

For this setup: leave it off unless the stream looks washed out or oddly bright and Windows HDR is on. To be tested on the capture monitors (open question: is HDR enabled on any of the three monitors?).

Note 2: Unity Setup

Unity is currently set up as a full-screen capture. That is suboptimal because of the constant switching between Unity and Visual Studio. Based on what has been learned about OBS, the plan is to change it to two separate scenes: one for Unity and one for Visual Studio.

Streaming Process and Workflow

Goal: exploit OBS much better so that moving between kinds of work is smooth and nearly automatic, for the viewer and for Robert. This matters most for class-creation streaming (the work on this course), where the stream has to jump constantly between many game engines, Claude, the browser, Acrobat and more. Today that switching is manual and ad hoc; the aim is a deliberate scene set and a repeatable routine.

Ideas to explore (not decisions yet):

- One scene per tool: a scene for each engine (and its play window), Claude, the browser, Acrobat, and the code editor, each capturing only that window, following the Note 1 rule of avoiding full-screen capture.
- Scene-switch hotkeys: one key per scene, so the switch before a play test (Note 1) costs no mouse travel.
- Combined scenes for common pairings: for example Claude beside an engine, or the browser beside Acrobat, so frequent back-and-forth does not need a switch at all.
- Transitions: a quick, consistent transition so scene changes look intentional rather than jarring.
- Starting and ending scenes: a short intro and a break/be-right-back scene, so pauses (and long Claude runs) look planned.
- Scene naming and ordering: name scenes by tool and keep the same order every session, so hotkeys and muscle memory stay stable.
- A written pre-stream checklist: open the tools, confirm each scene shows the right window, and check audio levels.
- Saved scene collections: one for class creation, one for general game development, so each setup can be recalled intact.

Workspace Layout: Three Monitors

Current setup, to design the OBS layout around:

Monitor	Position	Used for (currently)
Large monitor	Directly in front of Robert; to the right of the laptop monitor	Main work: development and play testing (the engines and their play windows, Visual Studio).
Laptop monitor	Slightly to Robert's left, beside the large monitor	Reference and supporting tools: Acrobat, Claude, the browser, and similar.
Small monitor	Above the laptop monitor, also slightly to the left	OBS itself, so it can be watched at a glance without covering any working window.

Goal: an OBS window layout optimized for this arrangement, so that Robert can run the stream, watch it, and switch scenes without ever moving a working window or dragging OBS over the work. Ideas to explore (not decisions yet):

- OBS lives full-screen on the small monitor, never on the large one: the large monitor stays 100% work and is where most scenes point.
- Use Studio Mode: the preview shows the scene about to go live and the program shows what viewers see. Switching becomes pick the scene, then transition, which suits the rule of switching BEFORE a play test.
- Dock only what is needed on the small screen: scene list, sources, audio mixer, and a stats panel (dropped frames and bitrate). Hide or float the rest so the scene list stays large and readable.
- Keep the chat dock on the small monitor too, if chat is being watched, so it does not take space on either working monitor.
- Scenes that capture the large monitor (engines, play windows) use window or game capture per Note 1. Scenes for the laptop monitor tools (Acrobat, Claude, browser) can use window capture of each tool, or a display capture of the laptop monitor if one scene covering all of it is simpler. Decide by testing which is less fragile.

- Consider a combined scene that shows an engine window from the large monitor beside a reference window from the laptop monitor, since class creation constantly pairs the two.
- Global hotkeys: OBS hotkeys can work while another window has focus, so scene switching does not require touching the OBS monitor at all; confirm this on the real setup.
- Mouse travel: the cursor crosses monitors to reach OBS. A hotkey set for the most common scenes avoids most of that travel.
- Watch which monitor shows what: take care that a display capture of the wrong monitor does not leak private windows onto the stream.

Go-Live Pre-Check Process

A short checklist to run before going live, and a lighter version before each scene transition. The aim is to catch failures such as the microphone dropout (see Note 3) before viewers do. Draft; adjust after trying it.

Before going live

- ☐ Open every tool the session needs (engines, Visual Studio, Claude, browser, Acrobat) and place each on its intended monitor.
- ☐ Click through each scene and confirm it shows the right window and nothing private (no wrong-monitor display capture).
- ☐ Confirm Windows Settings > System > Sound > Output is the HEADPHONES (not the laptop speakers) and that headphones are on, so the external microphone cannot pick up speaker sound.
- ☐ Speak and watch the microphone meter move; play a sound and watch the desktop audio meter move.
- ☐ Play something in each scene's engine or app and confirm that scene's application audio meter moves (Desktop Audio is not used; see Note 4).
- ☐ Confirm the microphone is not muted and is assigned to the track that is actually streamed (Advanced Audio Properties).
- ☐ Optional: turn on headphone monitoring for the microphone (headset icon beside its mute icon in the Audio Mixer) and listen briefly to confirm it is live; turn it off again if hearing your own voice is distracting (see Audio Monitoring below).
- ☐ Check the OBS stats panel: dropped frames, bitrate, CPU. Fix problems before starting.
- ☐ Confirm the chat dock, hotkeys and, if used, the recording are set up.
- ☐ Start with the intro scene so any settling or last-minute fixes happen off the main content.

Before each scene transition (especially before a play test)

- ☐ Switch to the scene for the window you are about to open, BEFORE launching the play test (Note 1).
- ☐ If using Studio Mode: look at the Preview, play a sound or speak, and confirm the preview scene's audio meters move (see Studio Mode and the Audio Mixer in Note 3), then transition.
- ☐ If not using Studio Mode: after switching, glance at the mic meter before talking.

During the stream

- ☐ Glance at the mic meter and the stats panel now and then; a flat mic meter while talking means act immediately.
- ☐ If audio drops: switch to the break scene, fix it, then return.

Note 3: Microphone Dropout Incident

On a recent stream the microphone cut out. Cause not yet identified. It was noticed too late because there was no way to see the microphone working before transitioning to a new scene. Wish: see that all audio is tracking in the Preview before transitioning.

Candidate causes to check, roughly from most to least likely:

- The microphone is a per-scene source rather than a global device. Robert uses the same microphone definition in all scenes (by accident, not design), so this cause is less likely. If it is a shared source, mute state and device carry across scenes. Optional hardening: assign the microphone under Settings > Audio > Mic/Auxiliary Audio so it is a global device.
- The microphone is on a different audio track than the one being streamed (Advanced Audio Properties).
- The source was muted in that scene.
- A device or driver hiccup in Windows, unrelated to scene changes. The OBS log (Help > Log Files > View Current Log) may show an error at the time.

Audio Monitoring (hearing a source in your headphones)

Robert's OBS shows two ways to control it:

- Each device in the Audio Mixer has a headset icon next to its mute icon, which enables or disables headphone monitoring for that device.
- Clicking the gear icon in the lower right of the Audio Mixer opens the advanced properties for that device. Its Audio Monitoring column has two choices, Monitoring Off or On.
- The headphones must be set as the monitoring device in Settings > Audio > Advanced > Monitoring Device. Use headphones, not speakers, or the microphone will pick up its own sound.

Effect: with monitoring on for the microphone, you hear your own voice in your headset, slightly delayed by OBS processing. Many people find that distracting, so monitoring is best used as a short check (is the mic live?) and then turned off, unless the delay does not bother you. Not yet confirmed: whether On here also keeps sending the source to the stream (it should, since the older Monitor and Output setting does). Check that the stream still carries the mic with monitoring on.

Studio Mode and the Audio Mixer (figured out)

- The Preview label appears only when the preview scene includes an audio source that is NOT in the active scene. A source that is in both scenes is marked Active.
- A preview-only source registers sound in the mixer while it is in Preview, so you can confirm it is working BEFORE transitioning. This is the ideal pre-transition check.
- FYI, possible exception: the first time a source is loaded into the preview during a stream, it may not register sound. To be tested (see Open Items).

Note 4: Audio Routing

Found while setting up: the laptop's internal speakers were the Windows output device, so the external microphone was picking up the speakers, and nothing played in the headphones. Desktop Audio in OBS only copies whatever Windows plays into the stream; it does not play anything to the headphones itself, because Windows does that.

Decisions (Robert):

- Stream with headphones on and set as the Windows output device, so the microphone never hears the speakers.
- Do not add Desktop Audio to the stream. Capture only the microphone and the audio of the window or application being shown (application audio capture, which is a beta feature in OBS and was tested and works).
- Do not turn on monitoring for any source that Windows already plays to the headphones (it would be heard twice).

Things to watch:

- Every sound source needs its own capture; an app with no capture is silent on the stream (a browser video, an app notification, each engine's play window).
- Application audio capture may lose its window when a play test is closed and restarted, the same weakness the Godot video capture had. Test per engine.
- If Desktop Audio is left in a scene or in the global list, mute it as a safety net so it can never double another source.

Capture Level vs Speaker Volume (believed, to be tested)

Observation (Robert): the game window's audio registered at about the same level in OBS regardless of the speaker volume. This matches how Windows audio capture generally works: OBS taps the sound before the Windows master volume is applied at the output, so the master volume slider should not change the level the stream receives. What does change it: the game's own volume setting, and possibly the per-app volume in the Windows Volume Mixer (not confirmed).

If true, the laptop speaker volume can be turned well down to reduce how much game sound the external microphone picks up, without changing the stream's game audio level. Test: play a sound, note the meter, lower the master volume, and see whether the meter holds steady.

Options: Headphones or No Headphones

Robert is still deciding whether to stream with or without headphones. Option summary (ideas, not decisions):

- A. Headphones on, set as the Windows output device. Removes speaker bleed completely. Costs: wearing them for long streams, and a little isolation from the room.
- B. Speakers, volume turned well down. Relies on the capture level not depending on speaker volume (test above). Some bleed may remain, depending on the microphone and where it sits.
- C. Speakers plus reducing what the microphone picks up: a directional (cardioid) microphone close to the mouth, positioned away from the speakers, and OBS microphone filters (Noise Gate, Noise Suppression) to cut quiet background sound between words. Worth testing as a complement to B.

- D. Headphones only during play tests. Take them off for talking-only segments if preferred; confirm the Windows output device switches back and forth cleanly (a pre-check item each time).

Why bleed matters even when the level seems low: the game audio is captured directly AND again, slightly later, through the microphone. Viewers can hear that as an echo or doubled sound, and it can muddy the speech. Headphones or low speaker volume avoid it.

Open Items

- Record the actual resolution and size of each monitor, and the OBS canvas and output resolution, so the layout can be tuned to real numbers.
- Sketch the OBS docks layout for the small monitor and save it as a named layout.
- Test whether a device that appears for the first time during a stream (first load into the Preview) registers sound in the mixer meter right away, or only after it has been loaded once. Note the OBS version (Help > About).
- Go through and validate sound levels: set and check the microphone level against the application audio in each scene (so neither drowns the other), confirm meters peak in a healthy range without clipping, and listen to a short test recording or stream before relying on the levels. Repeat for every scene that has a different application audio source.
- Check whether Windows HDR is on for any monitor; if so, test Force SDR on window sources and note the result.
- Test whether the Windows master volume affects the OBS application audio level (play a sound, change the volume, watch the meter). Then decide among the headphone options in Note 4.
- Test microphone filters (Noise Gate, Noise Suppression) for reducing speaker bleed if streaming without headphones.
- Test that application audio capture re-attaches after closing and restarting a play test, for each engine (Godot, GameMaker, Unity).
- Make the microphone a global audio device and delete per-scene copies; then retest scene switches for dropouts.
- Check the OBS log from the dropout stream for audio errors.
- Unity: build the two scenes and record the exact source types and settings that work.
- Other engines (Scratch, and any later additions): record the source type and settings once tested.
- Work through the workflow ideas above and record what is adopted; hotkeys look worth setting up, since the scene must change before every play test.